

CS253 Assignment 1: Memory-Efficient Versioned File Indexer

Dev

Roll Number: 240338

March 2026

1 Introduction and Overview

Large text files can cause excessive memory usage and performance bottlenecks if loaded entirely into memory. This project implements a memory-efficient versioned file indexer in C++. It reads text files incrementally using a fixed-size buffer, tokenizes the words in a case-insensitive manner, and stores their frequencies in a versioned index. It supports word count queries, top-K frequent word queries, and frequency difference queries between two versions.

2 Design and Approach

The system is built using object-oriented principles to ensure modularity. Instead of loading the entire file, the program allocates a strictly sized buffer (between 256 KB and 1024 KB) and reads the file in chunks. A tokenizer processes characters sequentially. To handle words that span across two consecutive chunks, a string boundary suffix is carried over to the next chunk reading. Word frequencies are stored efficiently in a hash map (`std::unordered_map`). The queries are dynamically executed utilizing an abstract base class, demonstrating C++ runtime polymorphism.

3 Description of the Classes Used

The solution utilizes four primary classes:

1. **BufferedReader**: Manages opening the file and iteratively reading chunk data into the fixed-size buffer. It validates that the buffer boundaries are adhered to.
2. **Tokenizer**: Processes the loaded character buffer. It builds alphanumeric words incrementally, converts them to lowercase, and registers them to the index. It natively handles words split across the buffer edge.
3. **VersionedIndex**: Stores the occurrences of each word associated with a specific file version. It implements function overloading for its `addWord()` methods.

4. **QueryProcessor**: An abstract base class that defines the pure virtual `execute()` function. It is inherited by three derived classes: `WordQuery`, `TopKQuery`, and `DiffQuery`, which handle specific command-line query operations using dynamic dispatch.

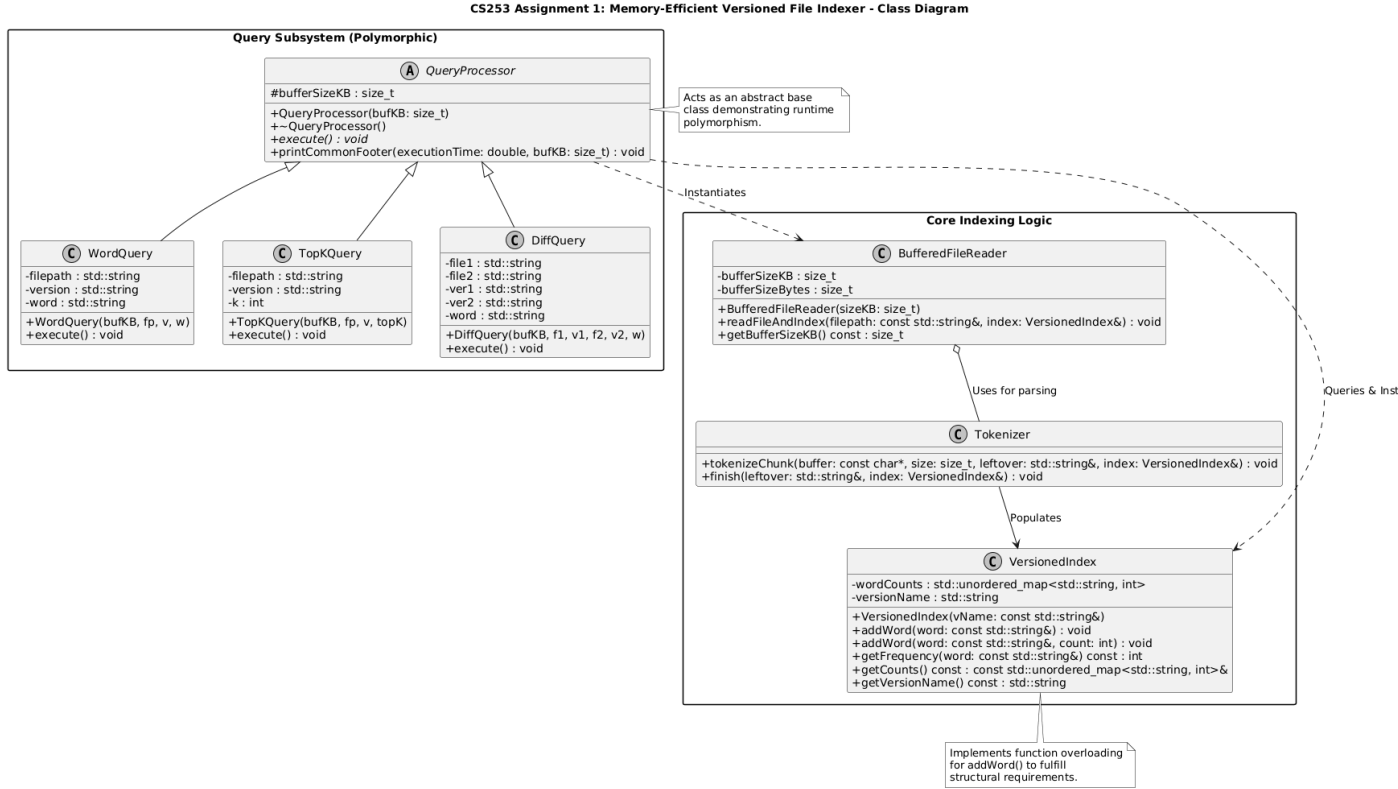


Figure 1: Class Diagram

4 Buffer Processing Methodology

File processing heavily relies on the `BufferedReader` and `Tokenizer` synergy.

- A `std::vector<char>` is allocated exactly to the requested bounds (256 KB to 1024 KB).
- The file is iteratively read using `file.read()`.
- In the `tokenizeChunk()` function, standard alphanumeric processing occurs. If a word is incomplete when the buffer iteration ends, it is saved in a `leftover` string variable.
- On the next buffer iteration, new characters for that token are appended to the `leftover` string until a non-alphanumeric character is encountered, resolving the boundary split correctly without losing data.

5 Assumptions and Observations

- **Words Definition:** Words are strictly defined as contiguous sequences of alphanumeric characters. Non-alphanumeric characters (like punctuation or spaces) explicitly serve as delimiters.
- **Case-Insensitivity:** The tokenizer forces all incoming characters to lowercase immediately, ensuring dictionary storage is completely case-agnostic.
- **Resource Scaling:** Memory overhead only scales relative to the number of *unique* words present across the file, as the dataset logic is purely frequency mapping rather than whole document storing.